

Topology-Informed Visual Prompting For Vision Language Action Policies

Anonymous Authors

Abstract—Vision-language-action (VLA) policies can struggle with manipulation tasks with complex obstacle geometries due to partial observability. These complex geometries can lead to similar visual observations or robot configurations requiring qualitatively different actions, a distinction that can be quantified using topological signatures. While motion planners with full knowledge of environment geometries and object states can reason about these signatures in planning, this information is often not known at deployment. To address this issue, we present a topology-guided visual-prompting framework that uses simulation-based planning to augment a nominal demonstration dataset and provides vision-based guidance at deployment. Our method uses a Gauss-Linking-Integral topological signature representation to capture important topological properties of the environment. Using privileged geometry information from a simulation approximation of our environment, we augment a VLA fine-tuning dataset with trajectories that move the system to a demonstrated signature and, from the new configuration, resume task execution. A vision-language model (VLM) is fine-tuned on the same dataset to both predict signatures from live camera observations and predict end-effector waypoints, which are rendered as visual prompts on the observations to guide the VLA. Across three simulated bimanual tasks and a real-world box pickup task, our method outperforms a VLA fine-tuned only on nominal demonstrations and a VLM-prompting baseline that can remove topology-relevant information from observations. On hardware, it exceeds the strongest baseline by 40% in task success. Project website: <https://topology-vla.github.io>.

Index Terms—Deep Learning Methods, Manipulation Planning

I. INTRODUCTION

Vision-language-action (VLA) policies have shown great promise for learning generalizable policies from diverse data sources. By combining large-scale Vision-Language Models (VLM) with action prediction, they aim to retain the generalizable intelligence of VLMs while adding action generation. An important question, however, is whether VLAs operating under partial observability can reliably solve manipulation problems that are traditionally addressed assuming full observability.

Given ground truth information about environment geometries and object states, model-based planners can explicitly reason about geometric and topological constraints, the latter of which is the focus of this work. Mitrano et al. [1] showed that for tasks where obstacle, robot, and environment geometries form closed loops, for example a robot grasping a cable threaded through part of the environment, as shown in Figure 2, explicitly reasoning about topological signatures can provide goal specifications and constraints for a planner. Specifically, they use the h-signature [2], a topological signature that describes how loops are linked.

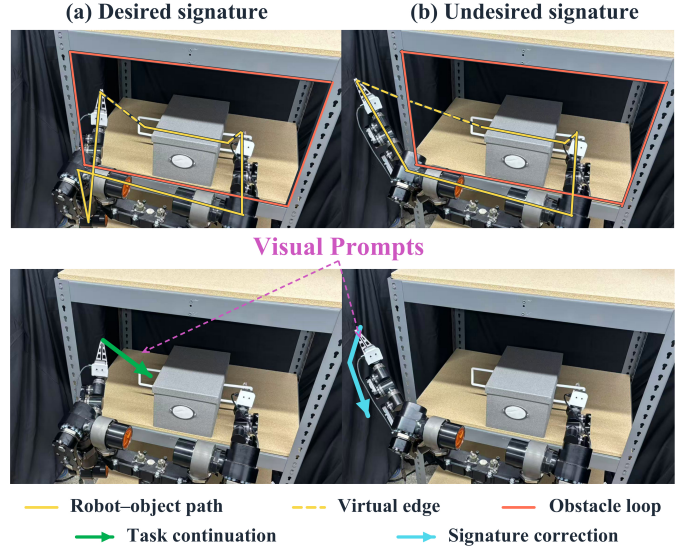


Fig. 1. Different topological signatures require different motion directions for the same task goal. With the demonstrated signature, our green visual prompt guides the robot directly toward the handle. With an incorrect signature, our cyan prompt guides it around the rack post before approaching the handle.

While more diverse demonstration collection could improve VLA performance without requiring explicit topological reasoning, fine-tuned policies can still fail. We frame our method as automating the augmentation of a nominal VLA fine-tuning dataset, enabling a large-scale automated version of data collection methods like DAGger [3] through planning in simulation. This allows us to take advantage of the capability of VLAs to operate under partial observability while still incorporating reasoning about topological signatures.

While VLA models do include VLM backbones that could be fine-tuned to support topology reasoning, prior work [4], [5] has shown degradation in vision-language reasoning abilities of the VLM backbones after action training. To account for this, we propose a framework that builds upon prior work in VLM prompting of VLAs [6].

We fine-tune a VLM to identify topological signatures from vision inputs and, if the signature differs from the demonstrated signature for the task, output a visual prompt to guide the VLA to recover to the correct signature. This visual prompt takes the form of an annotation drawn on the image that shows the needed end-effector motion to correct the signature.

The signature we use is a modification of the h-signature. To ensure the signature is defined even when loops are open, for example, before the robot grasps an object, we adapt the h-

signature formulation with *virtual links* that close open loops, as shown in Fig. 1. We refer to the signature computed from these virtually closed loops as the virtual h-signature.

Our contributions are:

- A method for fine-tuning a pre-trained VLM to perform topological signature reasoning.
- A visual-prompting framework that augments VLA inputs with topology-informed guidance, generated by a VLM.

We evaluate our method on multiple topologically complex bimanual manipulation tasks in simulation, and a box pickup task in the real world. In simulation and in the real world, our method improves success rate over a state-of-the-art VLM prompting baseline [6], which can inadvertently remove topologically-relevant information from VLA prompts. On hardware, our method outperforms the next best baseline by 40 percentage points.

II. RELATED WORKS

Our method expands upon prior work on augmenting VLAs through visual prompting, with those prompts being designed by a VLM. To fine-tune the VLM to generate useful prompts, we apply prior work on topological motion planning.

VLA Policies Recent work has explored large, generalist robot policies trained across diverse tasks and embodiments [7]–[9]. Building on this direction, vision-language-action (VLA) models leverage pretrained vision-language representations to connect semantic understanding with robot control [10]–[12]. In particular, π_0 combines a pretrained vision-language backbone with a flow-matching action expert to support general-purpose continuous robot control [12], while $\pi_{0.5}$ extends this through heterogeneous co-training across robot, semantic, and web data to improve generalization [13].

GR00T N1 couples a vision-language module with a diffusion-transformer action module for generalist humanoid control [14], MolmoAct introduces depth-aware perception and explicit spatial trajectory reasoning before low-level action prediction [15], and MolmoBot trains generalist manipulation policies on large-scale simulated demonstrations for zero-shot transfer to real robots [16].

However, these models still learn a direct mapping from visual observations, robot state, and language to actions. Nominal action supervision may not explicitly distinguish global relationships such as topology when locally similar inputs require different motions. In our experiments, we show that π_0 and $\pi_{0.5}$ models, even when fine-tuned on the tasks we consider, fail to learn generalizable topology reasoning but can be guided by our prompting framework.

VLM-Guided Visual Prompting Prior work uses image-space annotations to communicate spatial and motion information to vision-language models and robot policies. These annotations include demonstrated trajectories and motion traces, candidate actions for VLM-based selection, and compact visual cues that guide manipulation policies [6], [17]–[20]. These works demonstrate that visual prompts can provide an effective intermediate interface between high-level reasoning and robot control, motivating our approach to distill the VLM’s

topological reasoning into visual guidance for the VLA. However, they do not specifically consider topological signatures, leading to failures during task execution. We compare against PEEK [6], a method that attempts to mask out irrelevant parts of observations and draw a path for an end-effector to follow.

Topology-Guided Motion Planning Prior work has explored motion planning in topologically complex environments, using topological concepts such as homotopy or homology classes to inform search [1], [2], [21]. Other work uses topology to characterize robot–object interactions and manipulation states. Loop-based representations and quantities such as the Gauss linking integral have been used for grasping objects with holes, clasping and hooking, cooperative manipulation, and deformable-object manipulation [2], [22]–[27]. These works assume full knowledge of obstacle geometries at deployment time. In contrast, while we do assume access to a simulator in which we collect training data, our method does not require a high-fidelity simulation, allowing for discrepancies in obstacle geometries and layouts. Using this approximate simulation, we generate training data that can be deployed in evaluation environments which may differ from the training data generation environments.

III. PRELIMINARIES

The topological structures of interest are homotopy classes of task-relevant loops, defined in Mitrano et al. [1]. Two loops belong to the same homotopy class if one can be continuously deformed into the other without intersecting an obstacle. For manipulation, we construct loops from task-relevant robot, object, and obstacle geometry. We compute this signature by extending the h-signature, defined as the Gauss linking number between a task-relevant robot–object loop and a task-relevant obstacle loop. The linking number is an integer that measures the signed winding of the two closed loops through each other [2]. It is invariant to continuous deformation and can change only when loops pass through one another.

The physical robot and object paths in our tasks are open, so we extend the h-signature by connecting endpoints with virtual edges. As shown in Fig. 1, we first trace paths along the robot and manipulated object. Each robot path terminates at an end effector, which is paired with the endpoint of the object path at the corresponding grasp or contact site. These paths and endpoint pairings remain fixed across configurations of the task. At each configuration x_t , a straight virtual edge connects each paired endpoint. For example, for a bimanual grasp through two handles, the two handles are the endpoints of the object path, and the virtual edges connect them to the corresponding end effectors. The physical paths and virtual edges together form the closed robot–object loop L_t . We represent the task-relevant environmental structure by a separate closed obstacle loop O . The virtual edges represent candidate straight-line end-effector paths to their target contact sites. Including these paths in L_t lets the signature indicate whether straight-line motion is compatible with the demonstrated virtual h-signature or whether the robot must first route around an obstacle.

For a configuration x_t , its virtual h-signature $h^v = \text{Lk}(L_t, O)$, where Lk is the Gauss Linking integral as used in [2]. We define two configurations x and x' as having the same topological signature if and only if $h^v(x) = h^v(x')$. The same definition applies across tasks; only the physical paths, endpoint pairings, and obstacle loop used to construct L_t and O change with the task. This signature can therefore represent, for example, whether a cable passes through an aperture or whether a robot-object path is routed around an obstacle, independently of their poses and local geometry. A virtual edge might pass through an obstacle, for example, if an end-effector is below the rack shown in Fig. 1. Abstracting the scene into the L_t and O loops still allows us to compute h^v and use it to guide the policy, for example moving the end-effector above the rack to achieve the demonstrated h-signature.

IV. PROBLEM STATEMENT

We consider manipulation tasks whose success depends on achieving a specific h^v . Let $x_t \in \mathcal{X}$ denote the robot-object-environment configuration at time t , and let $h_t^v := h^v(x_t)$ denote its h^v -signature at time t . We denote the demonstrated task signature by $h^{v*} \in \mathbb{Z}$. Because a mismatch $h_t^v \neq h^{v*}$ can prevent direct progress toward the goal, successful execution may require first transitioning the system into the demonstrated signature. We assume that the task-relevant obstacle loop O is provided by a demonstrator. Given O and a demonstration, we can infer the demonstrated signature h^{v*} .

We assume access to a simulator of our tasks, in which we collect data. The simulator need not exactly reproduce the geometry and appearance of the deployment environment, instead serving as an approximate model for computing h^v values given O . O is only used to collect the simulation environment and is not assumed to be known in the real world. From the simulator, we collect an offline dataset

$$\mathcal{D} = \{\tau_i\}_{i=1}^N, \quad \tau_i = \{(o_t, s_t, h_t^v, a_t)\}_{t=0}^{T_i}.$$

Here, o_t is the visual observation, s_t is the robot state, and a_t denotes the expert action chunk beginning at time t . Each trajectory is also associated with a task instruction ℓ and demonstrated h-signature h^{v*} . The task-relevant obstacle geometry remains fixed within each rollout and is visible in o_t . Given $\mathcal{D}$, our goal is to learn a manipulation policy that completes the desired task. To do this, we need a framework that detects whether the current h^v matches the demonstrated h^v and corrects a mismatch when needed.

At deployment, h_t^v is not observed. We assume that o_t contains sufficient visual information to infer h_t^v . Nevertheless, configurations with similar local geometry can have different h^v and require qualitatively different manipulation strategies. Therefore, geometric proximity alone may not determine how the robot should progress toward the task goal. We evaluate a policy by its closed-loop task success rate, which measures whether it resolves h^v mismatches when necessary and completes the manipulation task.

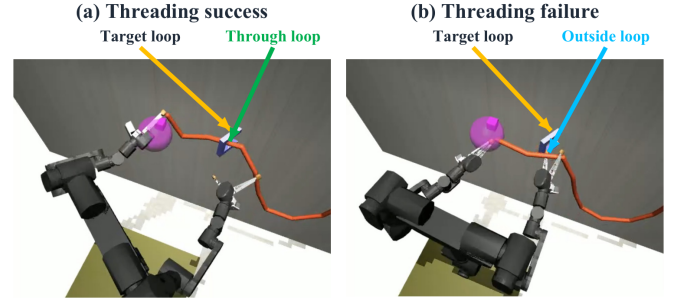


Fig. 2. Successful and failed cable-threading rollouts. In (a), the cable passes through the target loop before reaching the purple goal. In (b), π_{base} moves the cable tip toward the goal while leaving it outside the loop.

V. METHOD

A standard VLA maps visual observations, robot state, and a language instruction directly to robot motion. In our setting, this mapping is ambiguous because similar visual inputs under the same instruction can correspond to different signatures and require different motions. By augmenting the observation input to the VLA, our method resolves this ambiguity. At a high level, we use a VLM to reason about the h^v -signature and, based on this reasoning, provide visual prompts that guide the VLA. Figure 3 summarizes this architecture. We describe topology-aware demonstration generation in Sec. V-A, construction of the visual prompts in Sec. V-B, and VLM training and closed-loop guidance in Sec. V-C.

A. h^v -Aware Demonstration Generation

Our goal is to augment nominal demonstrations $\mathcal{D}_{\text{nominal}}$ with a correction dataset $\mathcal{D}_{\text{corr}}$ which moves configurations to the demonstrated h^v , and a continuation dataset $\mathcal{D}_{\text{cont}}$, which resumes task progress after correction, resulting in dataset

$$\mathcal{D}_{\text{train}} = \mathcal{D}_{\text{nominal}} \cup \mathcal{D}_{\text{corr}} \cup \mathcal{D}_{\text{cont}}. \quad (1)$$

We first train a base policy, π_{base} , by fine-tuning a pretrained VLA on a nominal demonstration dataset $\mathcal{D}_{\text{nominal}}$. These demonstrations cover successful task execution from randomized initial configurations, but they do not include explicit h^v annotations or targeted recovery behaviors for correcting h^v mismatches. As a result, π_{base} captures the nominal manipulation skills required for task completion, while leaving h^v -specific recovery behaviors underrepresented.

Being in the incorrect h^v could lead to a collision or block progress toward the goal. For example, as shown in Figure 2, a robot might move the cable tip toward the goal while leaving it outside the target loop. To augment the dataset with more diverse trajectories, we execute π_{base} in closed loop in simulation using the visual observation, robot state, and task instruction, and perturb states encountered along these rollouts. Privileged h^v labels from the simulator assign each perturbation resulting in a collision-free configuration to the appropriate recovery-data branch. The complete collection procedure is summarized in Algorithm 1.

At each collection step, PERTURBSTATE adds Gaussian noise to the current end-effector position, and a low-level

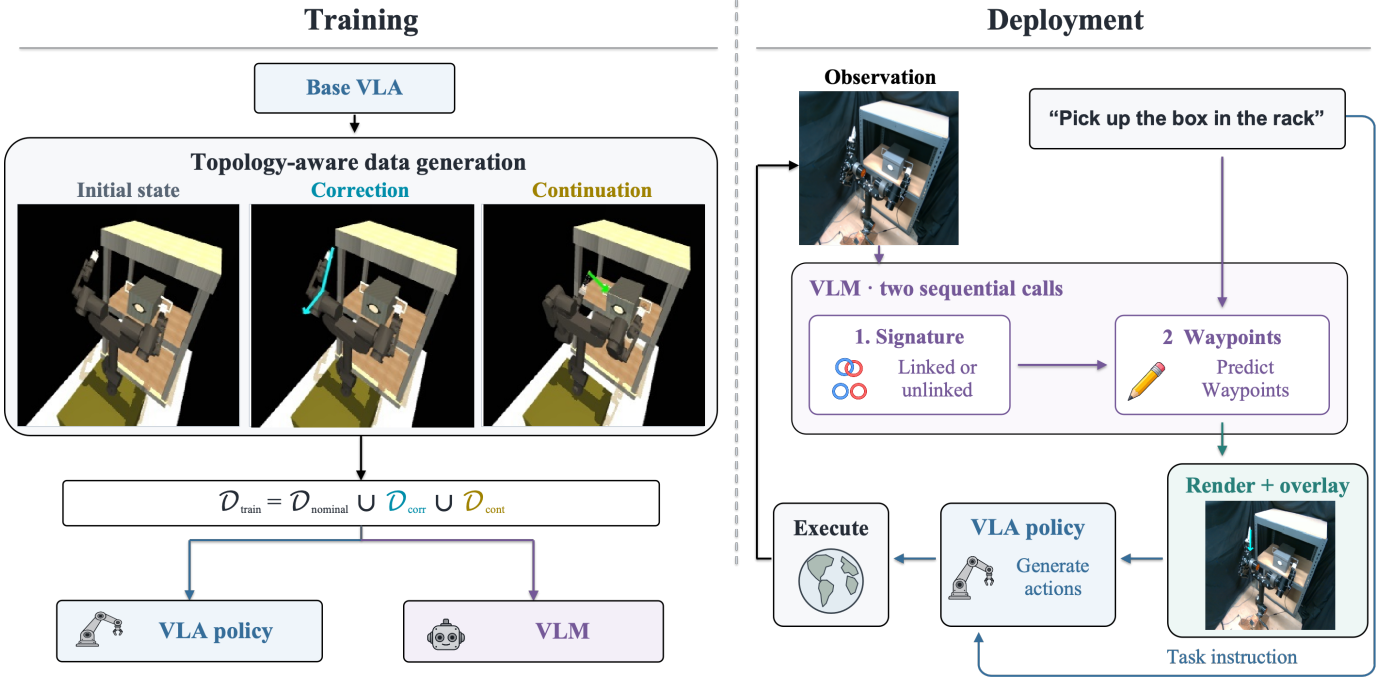


Fig. 3. Overview of the training and deployment pipelines. During training, h^v -correction and task-continuation trajectories augment the nominal demonstrations used to supervise the VLA and VLM. At deployment, the VLM first predicts the current h^v and then predicts image-space waypoints. At each policy update, these waypoints are rendered on the current observation to guide the VLA’s next action chunk.

Algorithm 1: h^v -Aware Demonstration Generation

Input: Base VLA π_{base} , simulator $\mathcal{E}$, demonstrated h -signature h^{v*}

Output: Correction data $\mathcal{D}_{\text{corr}}$ and continuation data $\mathcal{D}_{\text{cont}}$

```

 $\mathcal{D}_{\text{corr}}, \mathcal{D}_{\text{cont}} \leftarrow \emptyset$ 
foreach task instance do
  Initialize the task state  $x_0$ 
  while the nominal rollout is active do
    Execute  $\pi_{\text{base}}$  to obtain the next state  $x_t$ 
     $\tilde{x}_t \leftarrow \text{PERTURBSTATE}(x_t)$ 
    if  $\tilde{x}_t \neq \emptyset$  then
      Compute  $h^v(\tilde{x}_t)$  using the simulator
      if  $h^v(\tilde{x}_t) \neq h^{v*}$  then
         $\tau_{\text{corr}} \leftarrow \text{CORRECTSIGNATURE}(\tilde{x}_t, h^{v*})$ 
        if  $\tau_{\text{corr}} = \emptyset$  then
          continue
         $\mathcal{D}_{\text{corr}} \leftarrow \mathcal{D}_{\text{corr}} \cup \{\tau_{\text{corr}}\}$ 
         $\tau_{\text{cont}} \leftarrow \text{CONTINUE TASK}(x^+, \pi_{\text{base}}, h^{v*})$ 
        if  $\tau_{\text{cont}} \neq \emptyset$  then
           $\mathcal{D}_{\text{cont}} \leftarrow \mathcal{D}_{\text{cont}} \cup \{\tau_{\text{cont}}\}$ 
        else
          continue

```

controller moves the end effector toward the resulting target to produce $\tilde{x}_t$. This motion is used only to construct the perturbed configuration and is not in the training dataset. Unreachable targets are discarded. For each perturbation resulting in a

collision-free configuration, the simulator computes $h^v(\tilde{x}_t)$ and compares it with the demonstrated label h^{v*} .

When $h^v(\tilde{x}_t) \neq h^{v*}$, CORRECTSIGNATURE searches for a motion that is collision-free with respect to the physical robot, object, and environment and that changes the label to the demonstrated class. We sample candidate Cartesian targets around the active end effector and use the workspace-goal-directed RRT formulation of Vande Weghe et al. [28] to plan a collision-free joint-space trajectory to each target. Each resulting trajectory is executed in simulation and retained only if its terminal configuration x^+ satisfies $h(x^+) = h^{v*}$. When multiple valid corrections are found, we select the trajectory with the shortest end-effector path length.

After CORRECTSIGNATURE changes a mismatched configuration to a corrected state x^+ , this state may lie outside the distribution covered by the nominal demonstrations. CONTINUE TASK therefore executes π_{base} from x^+ and checks whether it reaches the next task subgoal while maintaining $h(x) = h^{v*}$. If the base VLA succeeds, CONTINUE TASK returns $\emptyset$, no continuation demonstration is added, and data collection proceeds to the next state of the nominal rollout. If the base VLA fails, a low-level end-effector IK controller generates a fallback continuation trajectory. We add only this fallback trajectory to $\mathcal{D}_{\text{cont}}$, provided that it reaches the subgoal while maintaining $h(x) = h^{v*}$. Otherwise, no continuation demonstration is added. If perturbation or correction fails, no demonstration is added for that candidate.

We train the final policy, π_{VLA} , as a separate fine-tuning run initialized from the original pretrained VLA rather than from π_{base} . All trajectories in $\mathcal{D}_{\text{train}}$ are pooled without balancing

or reweighting the three data sources. We describe how these end-effector trajectories are converted into visual prompts for VLM training below.

B. Visual Prompt Construction

We separate topology reasoning from action prediction because jointly adapting a VLM for reasoning and robot actions can degrade its pretrained reasoning capabilities [4], [5]. Following prior work on trajectory annotations [17], [18], we communicate h^v guidance through image-space drawings without modifying the VLA architecture.

We convert the end-effector trajectories generated in Sec. V-A into image-space drawings that are overlaid on the VLA’s visual observations. At each timestep, we project the desired end-effector motion onto the image from the scene camera. We denote the ordered image-space waypoint coordinates by W_t . A deterministic renderer connects these waypoints in temporal order and adds an arrowhead to indicate the direction of motion. We overlay this drawing on the visual observation o_t to obtain the prompted observation $\tilde{o}_t$.

We construct visual prompts for both h^v -correction and task-continuation trajectories. h^v -correction trajectories are rendered as arrows depicting the motion required to reach h^{v*} . These prompts consist of full trajectories and are capable of representing non-linear end-effector motion, for example a curved path around the post of a rack, by using multiple waypoints. Task-continuation trajectories are rendered as arrows in a different color from the topology-correction arrows, pointing toward the next task subgoal, such as a grasp location. Nominal task demonstrations are left unmodified. Each prompted observation $\tilde{o}_t$ is paired with the expert action chunk a_t , allowing the VLA to learn how image-space guidance maps to continuous control. The projected waypoint coordinates W_t also serve as supervision targets for the VLM described in the next subsection. Privileged quantities, including h_t^v and three-dimensional planning targets, are not provided to the policy.

C. Closed-Loop VLM Guidance

We train the VLA using drawings constructed from privileged h^v labels and expert trajectories, which are unavailable at deployment. We therefore train a single VLM to recognize the current h^v from camera observations and then predict the image-space waypoint coordinates. We separate these tasks because h^v labels are easier to obtain than expert trajectories.

We train the VLM with two types of supervised examples: 1) h^v -prediction examples pair a visual observation o_t with its simulator-provided label h_t^v ; and 2) Waypoint-prediction examples pair $(o_t, \ell, h_t^v, h^{v*})$ with the ordered image-space coordinates W_t constructed in Sec. V-B. Here, h_t^v provides the ground-truth current signature during training, and h^{v*} specifies the demonstrated signature. For nominal training states, the waypoint target is an empty sequence.

At deployment, we query the same VLM twice. We denote the h^v and waypoint calls by f_{h^v} and f_{way} . The first call predicts the virtual h-signature from the visual observation,

$$\hat{h}_t^v = f_{h^v}(o_t).$$

TABLE I
TASK SUCCESS RATES (%) IN SIMULATION.
BASE POLICY REFERS TO THE BACKBONE FINE-TUNED ON $\mathcal{D}_{\text{nominal}}$.

Task	Method	VLA backbone	
		π_0 [12]	$\pi_{0.5}$ [13]
Rope Pulling	Base Policy	70	52
	PEEK [6]	54	50
	Ours	66	64
Cable Threading	Base Policy	49	47
	PEEK [6]	13	20
	Ours	71	77
Box Pickup	Base Policy	31	45
	PEEK [6]	57	66
	Ours	84	82

Because the ground-truth current label is unavailable at deployment, the second call uses $\hat{h}_t^v$ with the visual observation, task instruction, and known demonstrated signature h^{v*} ,

$$\hat{W}_t = f_{\text{way}}(o_t, \ell, \hat{h}_t^v, h^{v*}).$$

The output is an ordered sequence of image-space waypoint coordinates. We render these coordinates as an arrow using the same visual conventions employed during VLA training, then overlay the resulting transparent layer on o_t to obtain $\tilde{o}_t$. An empty waypoint sequence leaves the observation unchanged. The VLA then maps the prompted observation, robot state, and original task instruction to a continuous action chunk,

$$\hat{a}_{t:t+H_a-1} = \pi_{\text{VLA}}(\tilde{o}_t, s_t, \ell),$$

where H_a is the action-chunk horizon. After executing each predicted chunk, we acquire a new observation and repeat both VLM calls. This closed-loop process updates the drawing along task progress.

VI. RESULTS

Our experiments address four questions. Does our method improve task performance across different h^{v*} values and VLA backbones, remain effective when only one h^v is possible, and transfer to real hardware? We evaluate Cable Threading and Box Pickup as tasks with multiple possible h^v values, Rope Pulling as a task with only a single possible h^v , and Box Pickup on hardware to assess real-world feasibility.

A. Evaluation Tasks

We consider three tasks in simulation: Rope Pulling, Cable Threading, and Box Pickup. For all tasks, we design simulation environments and task execution oracles to autonomously generate data for VLA and VLM fine-tuning.

1) *Rope Pulling*: Fig. 4(a) shows a representative initial observation. We evaluate each method over 50 trials with randomized rope, goal, and robot configurations. Because the rope tip begins beyond the robot’s reachable workspace, the robot alternates arms to grasp and pull it into the goal region. Since $h_t^v = 0$ throughout, no signature correction is required. This task tests whether our pipeline remains compatible with general learning without degrading nominal performance.

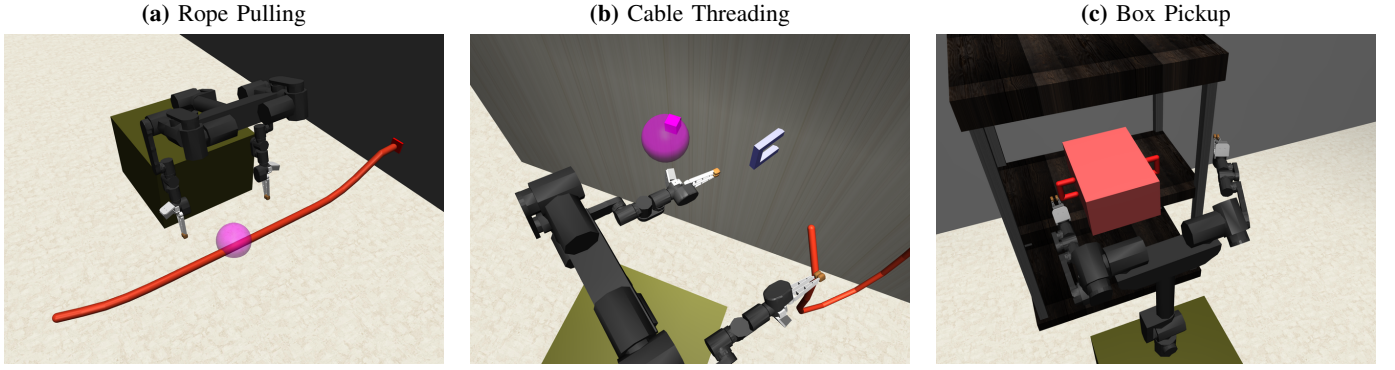


Fig. 4. Representative initial simulation observations for (a) Rope Pulling, (b) Cable Threading, and (c) Box Pickup.

2) *Cable Threading*: Fig. 4(b) shows a representative initial observation. We evaluate each method over 100 simulation trials. Cable Threading requires a transition from $h_0^v = 0$, where the cable has not passed through the target loop, to $h^{v*} = 1$. Its free tip is initially held by the right gripper on one side of the loop, with the left gripper near the opposite side. We randomize the cable configuration, robot configuration, and target-loop position across trials. Because moving the tip around the loop may bring it near the goal without achieving the required signature, the oracle passes the tip through the loop with the right gripper, secures it with the left gripper, and releases it with the right gripper. The left arm then pulls the tip toward the goal. A trial succeeds when the cable remains through the loop and its tip reaches the purple goal region without being dropped.

3) *Box Pickup*: Fig. 4(c) shows an initial observation in simulation. We evaluate each method over 100 simulation trials and 30 real-world trials. Box Pickup requires correcting $h_0^v = 1$ to $h^{v*} = 0$ before lifting the box. The box begins within a rack, with both grippers outside its two handle openings. We randomize the rack size, box position, and robot configuration across trials. In trials with $h_0^v = 1$, the right arm lies between the rack posts, so moving directly toward its handle would cause a collision. The robot must first route the arm around the posts to reach $h_t^v = 0$, then approach the handles, pass each gripper through its corresponding opening, and lift the box. A trial succeeds when the box clears the rack while maintaining the demonstrated signature and both grasps.

B. Implementation Details

We use π_0 [12] and $\pi_{0.5}$ [13] as VLA backbones and Qwen3-VL-8B-Instruct [29] for both VLM calls. For each backbone, the base policy, PEEK, and our method use the same VLA training configuration.

We fine-tune each VLA for 20,000 steps with a global batch size of 32, a learning rate of 5×10^{-5} , and 1,000 warm-up steps. We use a 10-step action horizon, 224×224 images, bfloat16 precision, and LoRA [30] ranks of 16 for the 2B vision-language backbone and 32 for the 300M action expert. Training uses four NVIDIA A40 GPUs. During execution, all 10 actions in a predicted chunk are completed before the next VLM and VLA update.

We fine-tune the VLM for h^v and waypoint prediction for one epoch using LoRA [30] with rank 8 and $\alpha = 16$. Training uses a global batch size of 32 across four NVIDIA A40 GPUs, a 4,096-token context, at most 262,144 image pixels, FP16 precision, and a cosine schedule with 10% warm-up. The learning rate is 5×10^{-5} for Box Pickup and 1×10^{-4} for Cable Threading.

C. Simulation Results

1) *Quantitative Results*: We use the same baselines and evaluation metrics for all three simulated tasks. For each VLA backbone, π_0 [12] and $\pi_{0.5}$ [13], we compare the base policy trained without data augmentation or additional annotations, PEEK [6], and our h^v -guided visual prompting. Table I reports task success rates for each method and backbone.

On Rope Pulling, our method performs comparably to the base policies and exceeds PEEK [6] by 12–14% across backbones, demonstrating compatibility with tasks with no signature correction. PEEK’s lower performance is likely because Rope Pulling repeatedly alternates arms that grasp and pull the rope, making it hard to separate the end effectors’ interleaved motions into subtrajectories.

On Cable Threading, our method exceeds the base policies by 22–30% and PEEK [6] by 57–58% across the two backbones. These gains support the proposed pipeline for execution requiring a transition to $h^{v*} = 1$. As in Rope Pulling, PEEK again struggles to distinguish the two arms’ subtrajectories during alternating motions. Here, execution must additionally respect the relationship between the robot–object and obstacle loops, making correct motion selection essential.

On Box Pickup, our method exceeds the base policies by 37–53% and PEEK [6] by 16–27% across the two backbones. Our method retains an advantage even when PEEK’s prompts benefit execution. Together with Cable Threading, these results support the pipeline’s applicability to tasks requiring different demonstrated signatures, including correction to $h^{v*} = 0$, rather than only a single type of transition.

To isolate signature recognition from task execution, we evaluate virtual h^v -signature prediction on 100 randomly sampled unseen configurations per task. Table II reports accuracy for each VLM trained on augmented data generated with its corresponding VLA backbone. Box Pickup accuracy is 16–18% higher than Cable Threading. This is consistent with

TABLE II
VLM PREDICTION ACCURACY (%) FOR THE VIRTUAL h^v -SIGNATURE ON
UNSEEN SIMULATION CONFIGURATIONS.

Task	VLA backbone	
	π_0 [12]	$\pi_{0.5}$ [13]
Cable Threading	74	76
Box Pickup	92	92

Box Pickup’s higher task success and suggests that accurate signature estimation contributes to performance.

2) *Qualitative Results*: Our method overlays topology-informed motion prompts on the original observation, preserving both the manipulation target and the surrounding obstacle geometries that constrain how it can be reached. The relevance of those geometries differs across tasks. Rope Pulling requires repeated grasping and pulling without modifying the topological signature. In Cable Threading and Box Pickup, however, the relationship between the robot–object loop and the obstacle loop determines whether the robot can approach the target directly or must first modify h^v .

The comparison with PEEK [6] illustrates the consequences of excluding this scene context. PEEK [6] emphasizes manipulated objects and subtrajectory goals but can mask geometry required for signature reasoning. Its masks omit the obstacle loop in Cable Threading and the relevant rack posts in Box Pickup, as shown in Fig.5(a). Our method preserves this geometry in Fig.5(b). Rope Pulling has no task-relevant obstacle loop and is consequently less sensitive to this loss of context, helping explain PEEK’s smaller performance drop. Together, these results suggest that visual guidance should preserve structures that constrain the motion, even when they are not themselves manipulation targets.

D. Real-World Results

To run the Box Pickup task on hardware, we apply visual domain randomization to the colors and textures of scene objects and to the camera positions and viewing angles. To address residual sim-to-real mismatch in reaching the box handles, we collect 15 short real-world trajectories demonstrating precise approaches to the handles for lifting the box. We also collect three h^v -correction trajectories to reduce the risk of overfitting to the reaching demonstrations. Both the baselines and our VLA are further fine-tuned on these 18 trajectories, whose total frame count is 3% of that in the collected simulation data.

For VLM adaptation, we human-label the h^v signature in observations from 1,000 random real-world configurations. We fine-tune the VLM on these labeled observations, which constitute approximately 5% of its total training data. During real-world execution, once the end effectors reach the designated handle regions, a low-level controller directly lifts the box. We do not fine-tune the PEEK VLM, as doing so would require full demonstrations on hardware which we do not assume we have. As such, our method does not rely on hardware teleoperation data which may be challenging to collect, especially for bimanual manipulation. At the same time, we can utilize cheaper-to-collect h^v -annotated data.

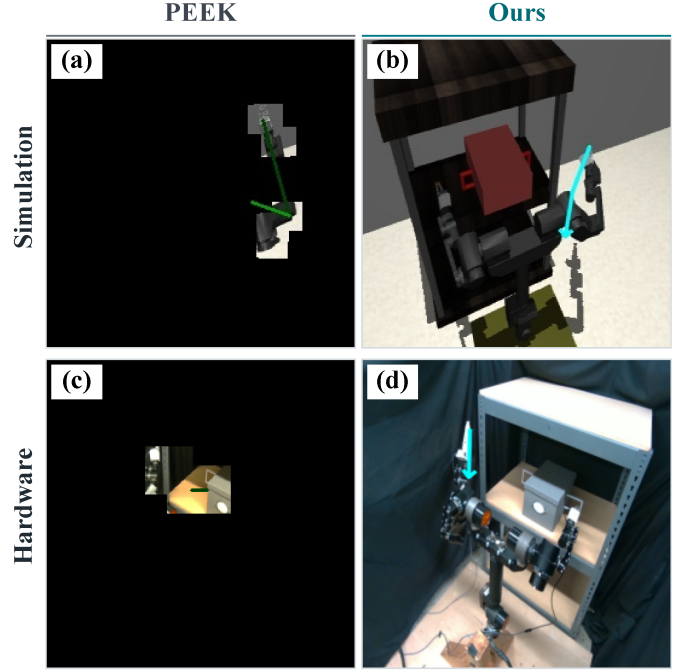


Fig. 5. Visual prompting for Box Pickup in simulation with (a) PEEK and (b) ours, and on hardware with (c) PEEK and (d) ours. PEEK masks out task-relevant rack geometry, whereas our method preserves this context and provides appropriate visual prompts

TABLE III
REAL-WORLD BOX PICKUP RESULTS. BASE POLICY REFERS TO THE VLA
BACKBONE FINE-TUNED ON $\mathcal{D}_{\text{nominal}}$.

Method	Overall task success (%)	h^v -correction success (%)
Base policy π_0 [12]	20	30
PEEK [6]	3	20
Ours	60	83

1) *Quantitative Results*: We report both overall task success and h^v -correction success. h^v -correction success measures whether the policy moves the robot from a configuration with an initial incorrect signature to a final configuration with the demonstrated signature. Under the real-world adaptation protocol described above, our method exceeds the strongest baseline by 40–53% on both metrics, as shown in Table III. The improvement is therefore not limited to signature correction but extends to task completion on hardware. For every method, correction success exceeds overall task success, showing that reaching the demonstrated signature does not by itself ensure a successful lift.

2) *Qualitative Results*: Fig. 5 compares the visual prompts produced by PEEK and our method. In configurations with an incorrect signature, the nominal VLA frequently reaches directly toward the handle without first moving around the rack post. This behavior resembles the direct approach demonstrated from training configurations with the demonstrated signature, but leads to obstruction in these real world initial states. The VLM trained using PEEK [6] similarly directs the end effector toward the handle without providing guidance around the post. In contrast, our method recognizes the in-

correct signature and guides the arm around the post before approaching the handle.

We also observe recovery from incorrect signature predictions during execution. In some trials, our method initially predicts the signature incorrectly and directs the end effector toward the handle. At a subsequent observation update, it revises the prediction and redirects the arm around the post. This illustrates how repeated signature prediction and visual-prompt updates can correct an initially inappropriate motion.

VII. CONCLUSION

We presented a topology-guided visual-prompting framework that combines planning in an approximate simulator with vision-based manipulation. Privileged simulator geometry supports the generation of signature-correction and task-continuation demonstrations for VLA fine-tuning. At deployment, a fine-tuned VLM predicts task-relevant signatures and end-effector waypoints, providing visual guidance without requiring full environment geometry. Across three simulated bimanual tasks and real-world box pickup, our method outperforms a VLM-prompting baseline and improves hardware task success by 40% over the strongest baseline. These results support using explicit signature reasoning to distinguish configurations requiring different motions, while preserving the scene context needed for execution.

REFERENCES

- [1] P. Mitrano and D. Berenson, "The grasp loop signature: A topological representation for manipulation planning with ropes and cables," in *ICRA*, 2024.
- [2] S. Bhattacharya, M. Likhachev, and V. Kumar, "Topological constraints in search-based robot path planning," *Autonomous Robots*, 2012.
- [3] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011.
- [4] A. J. Hancock, X. Wu, L. Zha, O. Russakovsky, and A. Majumdar, "Actions as language: Fine-tuning vlms into vlms without catastrophic forgetting," *arXiv preprint arXiv:2509.22195*, 2025.
- [5] S. Yang, H. Li, B. Wang, Y. Chen, Y. Tian, T. Wang, H. Wang, F. Zhao, Y. Liao, and P. Pang, "Instructvla: Vision-language-action instruction tuning from understanding to manipulation," *arXiv preprint arXiv:2507.17520*, 2025.
- [6] J. Zhang, M. Memmel, K. Kim, D. Fox, J. Thomason, F. Ramos, E. Bryk, A. Gupta, and A. Li, "Peek: Guiding and minimal image representations for zero-shot generalization of robot manipulation policies," *arXiv preprint arXiv:2509.18282*, 2025.
- [7] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu *et al.*, "Rt-1: Robotics transformer for real-world control at scale," *arXiv preprint arXiv:2212.06817*, 2022.
- [8] A. O'Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0," in *ICRA*, 2024.
- [9] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu *et al.*, "Octo: An open-source generalist robot policy," *arXiv preprint arXiv:2405.12213*, 2024.
- [10] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv preprint arXiv:2307.15818*, 2023.
- [11] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, "Openvla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.
- [12] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, " π_0 : A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.
- [13] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky, " $\pi_{0.5}$: a vision-language-action model with open-world generalization," 2025. [Online]. Available: <https://arxiv.org/abs/2504.16054>
- [14] NVIDIA, :, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. J. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llongtop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu, "Gr00t n1: An open foundation model for generalist humanoid robots," 2025. [Online]. Available: <https://arxiv.org/abs/2503.14734>
- [15] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee, W. Han, W. Pumacay, A. Wu, R. Hendrix, K. Farley, E. VanderBilt, A. Farhadi, D. Fox, and R. Krishna, "Molmoact: Action reasoning models that can reason in space," 2025. [Online]. Available: <https://arxiv.org/abs/2508.07917>
- [16] A. Deshpande, M. Guru, R. Hendrix, S. Jauhari, A. Eftekhari, R. Tripathi, M. Argus, J. Salvador, H. Fang, M. Wallingford, W. Pumacay, Y. Kim, Q. Pfeifer, Y.-C. Lee, P. Wolters, O. Rayyan, M. Zhang, J. Duan, K. Farley, W. Han, E. Vanderbilt, D. Fox, A. Farhadi, G. Chalvatzaki, D. Shah, and R. Krishna, "Molmob0t: Large-scale simulation enables zero-shot manipulation," 2026. [Online]. Available: <https://arxiv.org/abs/2603.16861>
- [17] J. Gu, S. Kirmani, P. Wohlhart, Y. Lu, M. G. Arenas, K. Rao, W. Yu, C. Fu, K. Gopalakrishnan, Z. Xu *et al.*, "Rt-trajectory: Robotic task generalization via hindsight trajectory sketches," *arXiv preprint arXiv:2311.01977*, 2023.
- [18] R. Zheng, Y. Liang, S. Huang, J. Gao, H. Daumé III, A. Kolobov, F. Huang, and J. Yang, "Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies," in *ICLR*, 2025.
- [19] Y. Li, Y. Deng, J. Zhang, J. Jang, M. Memmel, C. Garrett, F. Ramos, D. Fox, A. Li, A. Gupta *et al.*, "Hamster: Hierarchical action models for open-world robot manipulation," in *ICLR*, 2025.
- [20] S. Nasiriany, F. Xia, W. Yu, T. Xiao, J. Liang, I. Dasgupta, A. Xie, D. Driess, A. Wahid, Z. Xu *et al.*, "Pivot: Iterative visual prompting elicits actionable knowledge for vlms," *arXiv preprint arXiv:2402.07872*, 2024.
- [21] F. T. Pokorny, D. Kragic, L. E. Kavraki, and K. Goldberg, "High-dimensional winding-augmented motion planning with 2d topological task projections and persistent homology," in *ICRA*, 2016.
- [22] F. T. Pokorny, J. A. Stork, and D. Kragic, "Grasping objects with holes: A topological approach," in *ICRA*, 2013.
- [23] J. A. Stork, F. T. Pokorny, and D. Kragic, "A topology-based object representation for clasping, latching and hooking," in *Humanoids*, 2013.
- [24] A. Marzinotto, J. A. Stork, D. V. Dimarogonas, and D. Kragic, "Cooperative grasping through topological object representation," in *Humanoids*, 2014.
- [25] J. A. Stork, F. T. Pokorny, and D. Kragic, "Integrated motion and clasp planning with virtual linking," in *IROS*, 2013.
- [26] W. H. Lui and A. Saxena, "Tangled: Learning to untangle ropes with rgb-d perception," in *IROS*, 2013.
- [27] M. Sudry, T. Jurgenson, A. Tamar, and E. Karpas, "Hierarchical planning for rope manipulation using knot theory and a learned inverse model," in *CoRL*, 2023.
- [28] M. Vande Weghe, D. Ferguson, and S. S. Srinivasa, "Randomized path planning for redundant manipulators without inverse kinematics," in *Humanoids*, 2007.
- [29] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge *et al.*, "Qwen3-VL technical report," *arXiv preprint arXiv:2511.21631*, 2025.
- [30] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.